Lab Manual

Programming for AI Lab (AI2821)

Prepared by: Hammad Mustafa, Adan Atiq

Reviewed by: Adnan Karamat

Spring 2026

BS-AI

Department of Software Engineering

Capital University of Science and Technology, Islamabad

Contents

Lab 02: Python Basic Data Structures 3

1. **Introduction** 4
2. **Objective** 4
3. **Walkthrough Task** 4
   - **1. Strings** 4
   - **2. Slicing (Works for Strings and Lists)** 6
   - **3. Lists** 7
   - **4. Tuples** 9
   - **5. Sets** 10
   - **6. Dictionaries** 11
   - **Quick Summary** 13
4. **Practice Task:** 13
   - **Practice Task 01:** 13
   - **Practice Task 02:** 13
   - **Practice Task 03:** 13
5. **Conclusion** 14

Lab 02: Python Basic Data Structures

1. Introduction

Python provides several built-in data structures that help programmers store and manage data efficiently. Unlike languages such as C++, which often rely on arrays and manual memory management, Python uses flexible structures such as **strings, lists, tuples, sets, and dictionaries**. These structures allow programmers to easily manipulate text, store collections of values, manage unique data, and organize information using key-value pairs. Understanding these data structures is essential for writing efficient Python programs and for working in areas such as data science, artificial intelligence, and computer vision.

2. Objective

The objective of this lab is to help students understand the basic built-in data structures of Python. Students will learn how to create, access, and manipulate **strings, lists, tuples, sets, and dictionaries** using Python. By performing this lab, students will gain practical experience in handling different types of data and understand the differences between mutable and immutable data structures.

3. Walkthrough Task

This introduces some of the most important built-in data structures in Python:

- Strings
- Lists
- Tuples
- Sets
- Dictionaries

Understanding these structures is critical because Python programs rely heavily on them instead of manual memory management like C++ arrays.

1. Strings

A **string** is a sequence of characters used to store text.

Example:

```
name = "Hammad"
print(name)
```

Output:

```
Hammad
```

Accessing Characters

Strings support **indexing**.

```
name = "Hammad"
print(name[0])   # H
print(name[3])   # m
```

- Index starts from **0**.
- Important Concept: Strings are Immutable
- In Python, strings cannot be modified after creation.

Example (INVALID):

```
name = "Hammad"
name[2] = 'c'
```

This produces an error.

Why?

- Because Python strings are immutable, unlike C++ character arrays.
- Python prevents this to improve memory efficiency and safety.
- If you want to modify a string, you must create a new string.

Example:

```
name = "Hammad"
new_name = name.replace("m", "c")
print(new_name)
```

Useful String Functions

Length

```
text = "Python"
print(len(text))
```

Output:

```
6
```

Convert Case

```
text = "python"
print(text.upper())
print(text.lower())
```

Replace Text

```
text = "Hello World"
print(text.replace("World", "Python"))
```

Check Content

```
text = "python"
print(text.isalpha())
print(text.isdigit())
```

2. Slicing (Works for Strings and Lists)

Slicing

- Extracts a part of a sequence.

Syntax:

```
sequence[start:end]
```

Example:

```
text = "Programming"
print(text[0:4])
```

Output:

```
Prog
```

Note:

- start is included
- end is excluded

More Slicing Examples

```
text = "Programming"
print(text[:5])   # from start
print(text[3:])   # till end
print(text[-3:])  # last 3 characters
```

3. Lists

A **List** stores multiple values in a single variable.

Example:

```
numbers = [10, 20, 30, 40]
print(numbers)
```

Why Lists?

Because Python does not use arrays like C++ for most tasks.

C++ Equivalent

```
int arr[4] = {10,20,30,40};
```

Python Equivalent

```
numbers = [10,20,30,40]
```

But Python lists are more powerful.

They can store multiple data types.

```
data = [10, "hello", 3.14, True]
```

C++ arrays **cannot do this**.

Lists are Mutable

Unlike strings, lists **can be modified**.

```
numbers = [10,20,30]
numbers[1] = 99
print(numbers)
```

Output:

```
[10, 99, 30]
```

Important List Functions

Add Item

```
numbers = [10,20,30]
numbers.append(40)
print(numbers)
```

Output:

```
[10,20,30,40]
```

Insert at Position

```
numbers = [10, 20, 30]
numbers.insert(1, 15)
```

Ouput:

```
[10,15,20,30]
```

Remove Item

```
numbers.remove(20)
```

Pop (remove last element)

```
numbers.pop()
```

Length

```
len(numbers)
```

List Slicing

Lists support slicing exactly like strings.

```
numbers = [10,20,30,40,50]
print(numbers[1:4])
```

Output:

```
[20,30,40]
```

4. Tuples

A **Tuple** is similar to a list but **cannot be modified**.

Example:

```
data = (10,20,30)
print(data)
```

Tuple Properties

- Ordered
- Immutable
- Faster than lists
- Used when values should **never change**

Example:

```
coordinates = (10, 20)
```

Coordinates usually should not change.

Attempting to Modify a Tuple

```
data = (10,20,30)
data[1] = 50
```

This produces an **error**.

Why Tuples are Faster

Because Python knows values cannot change, it performs fewer safety checks.

Therefore they are more memory efficient and faster.

Use tuples when:

- Data is fixed
- You want performance improvement
- You want to prevent accidental modification

5. Sets

A **Set** is a collection of **unique values**.

Example:

```
numbers = {10,20,30,20,10}
print(numbers)
```

Output:

```
{10,20,30}
```

Duplicate values are automatically removed.

Difference Between List and Set

Feature	List	Set
Duplicates	Allowed	Not allowed
Order	Maintained	Not guaranteed
Indexing	Supported	Not supported

Useful Set Functions

Add Value

```
numbers.add(40)
```

Remove Value

```
numbers.remove(20)
```

Set Union

```
a = {1,2,3}
b = {3,4,5}
print(a.union(b))
```

Output:

```
{1,2,3,4,5}
```

6. Dictionaries

A **Dictionary** stores data in **key-value pairs**.

Example:

```
student = {
    "name": "Hammad",
    "age": 21,
    "cgpa": 3.8
}
```

Access Values

```
print(student["name"])
```

Output:

```
Hammad
```

Modify Values

```
student["age"] = 22
```

Add New Key

```
student["city"] = "Rawalpindi"
```

Important Dictionary Functions

Get Value

```
student.get("name")
```

Get All Keys

```
student.keys()
```

Get All Values

```
student.values()
```

Quick Summary

Data Type	Mutable	Ordered	Unique
String	No	Yes	No
List	Yes	Yes	No
Tuple	No	Yes	No
Set	Yes	No	Yes
Dictionary	Yes	Yes	Keys unique

4. Practice Task:

Practice Task 01:

Scenario: Username Formatting System **Expected time 20 mints**

A university IT department is creating official email accounts for students. However, students often enter their usernames in inconsistent formats such as uppercase letters, lowercase letters, or with extra spaces. Write a Python program that asks the user to enter several usernames and stores them in a list. The program should clean the usernames by removing extra spaces and converting them to lowercase. After processing, display the cleaned usernames, print the first three characters of each username, and show the total number of usernames entered.

Practice Task 02:

Scenario: Online Shopping Cart **Expected time 20 mints**

An online store wants to simulate a simple shopping cart system for customers. Write a Python program that stores product names added to a customer's cart in a list. The program should allow the user to add new products to the cart and remove a product if the customer decides not to purchase it. After performing these operations, display all products currently in the cart, show the total number of products in the cart, and display a portion of the cart containing only the first three products.

Practice Task 03:

Scenario: Student Marks Record **Expected time 20 mints**

A teacher wants to maintain a small digital record of students and their marks. Write a Python program that stores student names and their marks using a dictionary. The program should display the marks of a specific student, allow the teacher to update the marks of a student if needed, and add a new student with their marks to the record. Finally, display all student names along with their marks stored in the system.

5. Conclusion

In this lab, students explored the fundamental data structures in Python including **strings, lists, tuples, sets, and dictionaries**. They learned how each structure stores data and how they differ in terms of mutability, ordering, and uniqueness. Through practical examples and exercises, students gained hands-on experience in manipulating these structures, which are essential for building efficient Python programs and understanding advanced topics in programming and data analysis.